A Zero-BRAM, Single-Cycle Streaming 2D Convolution Accelerator with Saturating ReLU Datapath for Edge-AI Vision

Ahmed S. Bassala[1], *Student Member, IEEE*
[1]Department of Electronics and Communications Engineering, Egypt-Japan University of Science and Technology (E-JUST)

Competition Track: IEEE SSCS Egypt Chapter 2026 Student Design Competition
Kernel Specification: $N = 3$ (Single 3×3 Spatial Filter, Runtime-Configurable Weights)
Architectural Highlight: Distributed SRL32 Shift-Register Line Buffering with 100% Valid Output Streaming Throughput

***Abstract*—This paper presents the digital architecture, hardware implementation, and systematic verification of an area- and energy-efficient 2D convolutional accelerator tailored for edge-AI vision applications. Operating on single-channel 32×32 image frames, the accelerator evaluates a runtime-programmable 3×3 spatial kernel with 8-bit unsigned pixel activations and 8-bit signed weights. By implementing line buffers via look-up table shift registers (SRL32), Block RAM utilization is strictly eliminated (BRAM $= 0$), avoiding severe Figure-of-Merit (FoM) penalties. The pipelined datapath couples a parallel LUT-based multiplier array with a balanced combinational adder tree and saturating post-processing with optional Rectified Linear Unit (ReLU) activation. Implemented Out-of-Context on a Xilinx Artix-7 FPGA (`xc7a35tcpg236-1`), the core delivers a sustained throughput of 1.0 output pixel/clock cycle after an initial latency of 69 cycles, closing at 100.01 MHz, and achieving an on-chip power consumption of 75 mW (0.075 W), and an optimized Figure of Merit of 0.01734 pixel$/(\mathrm{W} \cdot \mathrm{Cost})$.**

***Index Terms*—Convolutional Neural Networks, 2D FIR Filter, FPGA, Edge-AI, Line Buffers, SRL32, Saturation Arithmetic, Figure of Merit.**

I. Specifications and Architectural Assumptions

To satisfy all mandatory contest requirements while establishing rigorous bounds for hardware sizing, the parameters in Table I govern the design.

TABLE I
System Specifications

Parameter	Design Choice	Justification
Input Resolution	32×32 pixels	Minimum spec-compliant resolution.
Input Precision	8-bit Unsigned ($[0, 255]$)	Native dynamic range of monochrome image sensors (standard grayscale range).
Kernel Dimensions	3×3 ($N = 3$)	Dominant spatial filter size across modern convolutional neural network backbones.
Kernel Precision	8-bit Signed ($[-128, 127]$)	Specification.
Stride	$S = 1$	Specification.
Output Precision	16-bit Signed	Minimum spec-compliant precision.
Boundary Policy	Valid-Region (30×30)	Avoids artificial zero-padding distortion.
Activation Function	Saturating ReLU	Bonus feature; implements non-linear activation $f(x) = \max(0, x)$ at almost zero logic overhead.

II. Architecture Overview

The accelerator is partitioned into six distinct structural units, as illustrated in the top-level block diagram (Fig. 1):

1) Configuration Port: A 4-bit addressed write interface (`cfg_we`, `cfg_addr`, `cfg_wdata`) that loads weights into the internal 3×3 coefficient matrix prior to frame execution.
2) Dual Line Buffers (`lb0`, `lb1`): Shift register storage buffering $2 \times (W - 3)$ pixels, aligning three consecutive vertical image rows.
3) Sliding Window Generator: A 3×3 flip-flop array that captures spatial neighborhoods on every active clock cycle.
4) Parallel Multiplier Core: Nine parallel fabric multipliers calculating signed products between unsigned pixels and signed weights without dedicated DSP slices (`(* use_dsp = "no" *)`).
5) Balanced Adder Tree: A 4-level combinational tree accumulating 9 products into an overflow-free 20-bit signed sum (`acc_comb`).
6) Saturating Post-Processor: Dual-threshold comparator clamping values to signed 16-bit limits, cascaded into an optional sign-bit ReLU gate.

III. Datapath and Bit-Width Analysis

A. Multiplier Dynamic Range

The input pixels are unsigned values $X_{i,j} \in [0, 255]$ represented as `logic [7:0]`. In the multiplier array, each pixel is concatenated with a leading zero to cast it into a non-negative 9-bit signed integer:

$$X_{\text{ext}} = \$signed(\{1'b0, win[r][2-c]\}) \in [0, +255] \quad (1)$$

The programmable kernel taps $W_{m,n} \in [-128, +127]$ are 8-bit signed two's complement integers sign-extended to 9 bits. The resulting product dynamic range is:

$$P_{\min} = 255 \times (-128) = -32,640 \quad (2)$$

$$P_{\max} = 255 \times (+127) = +32,385 \quad (3)$$

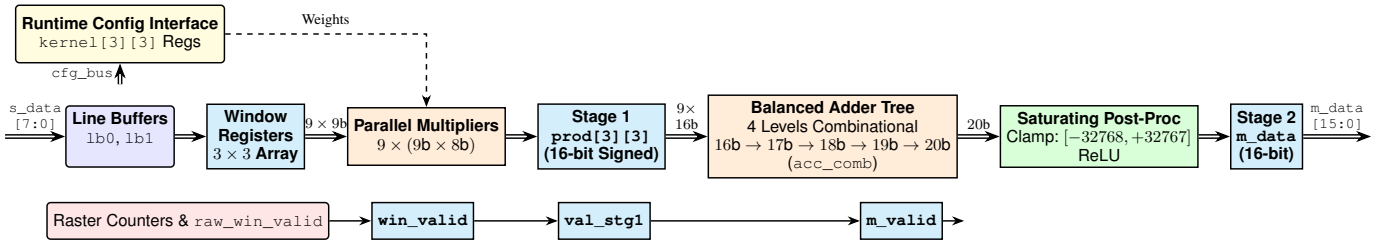


Fig. 1. Complete hardware architecture and pipelined datapath of the convolution accelerator, detailing signal bit-growth and synchronous valid-strobe tracking.

Because $-2^{15} = -32,768 \leq P_{\min} < P_{\max} \leq +32,767 = 2^{15} - 1$, every individual product strictly fits within a standard 16-bit signed container. In RTL, the multiplier outputs are registered directly as `logic signed [15:0] prod[3][3]`.

B. Adder Tree Dynamic Range

The accumulator sums nine parallel 16-bit signed products:

$$S_{\text{acc}} = \sum_{r=0}^{2} \sum_{c=0}^{2} P_{r,c} \tag{4}$$

The theoretical worst-case accumulation extrema evaluate to:

$$S_{\min} = 9 \times (-32,640) = -293,760 \tag{5}$$

$$S_{\max} = 9 \times (+32,385) = +291,465 \tag{6}$$

The minimum bit-width required to contain this complete sum without arithmetic overflow is:

$$B_{\text{acc}} = 1 + \lceil \log_2(293,760) \rceil = 1 + 19 = 20 \text{ bits} \tag{7}$$

The datapath implements an exact 20-bit signed accumulator (`logic signed [19:0] acc_comb`) spanning $[-524,288, +524,287]$. This provides full mathematical protection against intermediate overflow while maintaining optimal slice logic utilization.

TABLE II
BIT-WIDTH PROGRESSION ACROSS DATAPATH STAGES

Stage / Signal	Representation	Bit Container	Absolute Dynamic Range
Input Pixel (`s_data`)	Unsigned	`logic [7:0]`	$[0, 255]$
Sign-Ext Pixel (`win`)	Signed Positive	`logic signed [8:0]`	$[0, +255]$
Kernel Weight (`kernel`)	Signed 2's Comp	`logic signed [7:0]`	$[-128, +127]$
Product $P_{r,c}$ (`prod`)	Signed 2's Comp	`logic signed [15:0]`	$[-32640, +32385]$
Adder Level 1 (`lvl1_sum*`)	Signed 2's Comp	`logic signed [16:0]`	$[-65280, +64770]$
Adder Level 2 (`lvl2_sum*`)	Signed 2's Comp	`logic signed [17:0]`	$[-130560, +129540]$
Adder Level 3 (`lvl3_sum0`)	Signed 2's Comp	`logic signed [18:0]`	$[-261120, +259080]$
Adder Level 4 (`acc_comb`)	Signed 2's Comp	`logic signed [19:0]`	$[-293760, +291465]$
Clamped Output (`sat_val`)	Signed 2's Comp	`logic signed [15:0]`	$[-32768, +32767]$
ReLU Output (`m_data`)	Signed 2's Comp	`logic signed [15:0]`	$[0, +32767]$

IV. MEMORY ORGANIZATION AND WINDOW GENERATION

A. Line Buffer Depth Derivation

In raster-scanned 2D convolution, pixels arrive sequentially row-by-row. To expose a 3×3 kernel simultaneously, the hardware must maintain three vertically aligned taps: (r, c), $(r-1, c)$, and $(r-2, c)$.

Because each row streams across three horizontal registers inside the 3×3 window (`win[r][0]` $\rightarrow$ `win[r][1]` $\rightarrow$ `win[r][2]`), each row is delayed by 3 clock cycles in flip-flops before feeding into the line buffer FIFO. Consequently, the required intermediate delay line depth is derived as:

$$D_{\text{LB}} = W - N = 32 - 3 = 29 \text{ words} \tag{8}$$

Total row-to-row traversal latency is:

$$\begin{aligned} T_{\text{row}} &= 3 \text{ (window registers)} \\ &\quad + 29 \text{ (line buffer FIFO)} = 32 \text{ clock cycles} \end{aligned} \tag{9}$$

This perfectly aligns rows r, $r-1$, and $r-2$ across consecutive lines.

B. LUT Shift-Register (SRL32) Architecture

Standard FPGA implementations often map line buffers into Block RAMs. However, the contest Figure of Merit imposes a heavy penalty of $100 \times$ BRAMs.

For an image width of 32, a 29-entry 8-bit FIFO requires only $29 \times 8 = 232$ bits of storage. Instantiating an 18-kbit or 36-kbit BRAM primitive would waste $> 98\%$ of its capacity while severely degrading the FoM score.

By isolating the line buffer arrays into a dedicated clock process without an asynchronous reset and applying the synthesis pragma:

```
(* srl_style = "srl" *)
logic [DATA_WIDTH-1:0] lb0 [LB_DEPTH];
(* srl_style = "srl" *)
logic [DATA_WIDTH-1:0] lb1 [LB_DEPTH];
```

the synthesis engine packs the FIFO chains directly into native **SRL32E primitives** (LUTs configured as 32-bit shift registers). As verified in the post-implementation utilization summary, the design consumes exactly 16 LUTs as Memory with strictly zero Block RAM blocks.

C. Raster Coordination and Boundary Masking

The internal window coordinate is tracked by two counters:

$$\text{col_cnt} \in [0, W-1] \tag{10}$$

$$\text{row_cnt} \in [0, H-1] \tag{11}$$

The boundary qualification logic is implemented combinatorially and registered to decouple timing:

$$\text{raw_win_valid} = \text{s_valid} \wedge (\text{row_cnt} \geq 2) \wedge (\text{col_cnt} \geq 2) \tag{12}$$

$$\text{win_valid} \Leftarrow \text{raw_win_valid} \tag{13}$$

This condition accomplishes two objectives:

1) Warm-Up Blanking: Prevents invalid calculations during the first 66 cycles while line buffers and window registers fill.
2) Row Boundary Blanking: Deasserts qualification for 2 cycles at column indices 0 and 1 of every row, preventing the sliding window from wrapping across adjacent lines.

Across a full 32×32 frame, valid calculations occur for exactly $(32-2) \times (32-2) = \mathbf{900}$ clock cycles.

V. Control Strategy

The accelerator intentionally foregoes a traditional multicycle algorithmic state machine (FSM). In high-throughput streaming datapaths, classical FSMs introduce control state decode overhead, branch delay hazards, and unnecessary flip-flop toggling.

Instead, the architecture implements a decoupled valid-strobe pipeline:

- Configuration Phase: Handled synchronously using `cfg_we`. Weights update when `cfg_we` is high and `cfg_addr` < 9.
- Execution Phase: Self-synchronizing streaming datapath where `s_valid` clocks the line buffers, window registers, and coordinate counters.
- Latency Tracking: The qualifier `raw_win_valid` passes through `win_valid` and `val_stg1`, emerging as `m_valid` aligned with the computed pixel.

VI. Pipeline Structure and Latency Analysis

The complete datapath spans four sequential register stages:

- *Stage 0 (Buffer / Window Capture):* Serial input stream `s_data` shifts into `win[2][0]` and traverses dual SRL lines `lb0` and `lb1`. The bottom-right corner of the first valid 3×3 receptive field enters at coordinate $(\text{row_cnt} = 2, \text{col_cnt} = 2)$, asserting `raw_win_valid` combinatorially.
- *Stage 1 (Window Qualification Register):* The boundary qualification strobe is registered into `win_valid` via `stage0_valid` to eliminate combinational path propagation into the arithmetic core.
- *Stage 2 (Parallel Multiplier Array):* Pixels in `win[3][3]` are multiplied with kernel weights, registering products into `prod[3][3]` alongside pipeline valid flag `val_stg1`.
- *Stage 3 (Adder Tree, Saturation, ReLU & Output Register):* A 4-level balanced combinational adder tree evaluates Levels 1 through 4 into `acc_comb`, performs saturation clamping to signed 16-bit bounds with optional ReLU gating, and latches the final result into `m_data` while asserting `m_valid`.

A. Latency Budget

The initial latency required to emit the first valid feature-map pixel is formulated as:

$$\begin{aligned} L_{\text{fill}} &= 2 \times W + 2 = 2(32) + 2 = 66 \text{ cycles} \\ L_{\text{pipe}} &= 3 \text{ cycles} \\ &\quad (\texttt{win_valid} + \texttt{prod} + \texttt{m_data} \text{ registers}) \\ L_{\text{total}} &= 66 + 3 = 69 \text{ clock cycles} \end{aligned} \tag{14}$$

As verified in functional simulation (Fig. 2), the active-high strobe `m_valid` asserts exactly at clock cycle 69 (coinciding with coordinate counter col_cnt $= 5$ on row 2). Following this deterministic warm-up latency, the core sustains a continuous throughput of 1.0 output pixel / cycle for all interior window coordinates.

VII. Fixed-Point and Overflow Handling

A. Saturation Logic

Direct truncation of multi-operand accumulators induces severe wrap-around artifacts (positive peaks overflowing into negative valleys). To preserve feature fidelity, hardware clamping enforces saturation boundaries over the 20-bit accumulator `acc_comb`:

$$\text{Sat}(S) = \begin{cases} +32,767 & \text{if } S > +32,767 \\ -32,768 & \text{if } S < -32,768 \\ S[15:0] & \text{otherwise} \end{cases} \tag{15}$$

B. Bonus Feature: ReLU Activation

The Rectified Linear Unit (ReLU) activation function:

$$f(x) = \max(0, x) \tag{16}$$

is evaluated in the combinational post-processing block. The sign bit of the clamped 16-bit value (`sat_val[15]`) controls the zero-multiplexer:

```
if (enable_relu && sat_val[15])
    post_proc_val = 16'sd0;
else
    post_proc_val = sat_val;
```

When enabled, this stage replaces negative activations with zero while passing positive values without adding extra clock latency.

VIII. Verification Methodology

Verification was conducted using a co-simulation environment comparing the SystemVerilog RTL against a bit-accurate Python NumPy golden reference model.

A. Test Case Selection

The verification plan was designed by selecting specific test cases to test distinct boundary conditions as well as test normal behavior across the RTL datapath:

1) Tested reg write with randomized stimulus to ensure kernel registers latch correct coefficient addresses.

2) Identity 3×3 kernel to validate zero-skew in line-buffer indexing, correct spatial alignment, correct raster coordinate count, and correct pipeline delay.
3) Sobel X and Y edge detection kernels were used to test behavior with signed-value kernels as well as correct datapath logic.
4) Laplacian operator kernel was tested with dark to bright transition to test accumulation of negative values as well as ReLU behavior.
5) Maximum dynamic range was tested with maximum weight kernel to test that the adder tree accumulates results without overflow and with saturation at $+32,767$.
6) Correct number of valid output pixels (900) and boundary blanking was tested by comparing total `m_valid` pulses across the entire frame to 900.

IX. Testbench Architecture and Waveforms

The self-checking SystemVerilog testbench (`tb_conv3x3.sv`) handles stimulus generation and response monitoring:

1) Reads input pixel frames (`pixels_in.hex`), golden reference outputs (`golden_out.hex`), and kernel configurations (`weights.hex`) via `$readmemh`.
2) Emulates runtime programming over `cfg_bus`.
3) Drives continuous raster streaming at 1 pixel/cycle.
4) Operates a scoreboard comparing `m_data` against golden arrays on every rising clock edge where `m_valid` is asserted.

X. FPGA Implementation Results

To evaluate the intrinsic hardware efficiency of the accelerator datapath, characterization was performed Out-of-Context (OOC) using AMD Vivado targeting a standard Xilinx Artix-7 FPGA (`xc7a35tcpg236-1`). Timing analysis was evaluated under a nominal 100 MHz target clock constraint ($T_{\text{clk}} = 10.0$ ns). This isolates the core arithmetic pipeline from chip-level I/O pad parasitics and unconstrained board pin penalties, providing a clean benchmark. Implementation was carried out on Vivado 2018 using `Performance_ExplorePostRoutePhysOpt` strategy.

TABLE IV
FPGA Post-Implementation Resource and Performance Summary

Resource / Metric	Utilization	Available	Percentage
Slice LUTs	769	20,800	3.70%
Slice Registers (FFs)	317	41,600	0.76%
Occupied Slices	238	8,150	2.92%
LUT as Memory (SRL)	16	9,600	0.17%
DSP48E1 Blocks	0	90	0.00%
Block RAM (RAMB18/36)	0	50	0.00%
Worst Negative Slack (WNS)	+0.001 ns	—	Setup Met
Worst Hold Slack (WHS)	+0.147 ns	—	Hold Met
Maximum Frequency ($F_{\max}$)	100.01 MHz	—	Met at 100 MHz
Total On-Chip Power	0.075 W	—	Vectorless Pwr
Dynamic On-Chip Power	0.007 W	—	9% of Total
Device Static Power	0.068 W	—	91% of Total

XI. Design Trade-Offs and Architectural Analysis

A. SRL Shift Registers vs. Dedicated Block RAM

The competition Figure of Merit evaluates resource cost as:

$$\text{Cost} = \text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs} \tag{17}$$

A naive implementation assigning line buffers to dual-port RAM primitives consumes at least 2 BRAM blocks ($2 \times 100 = 200$ equivalent LUTs). By shifting storage into distributed LUT-RAM (SRL32E), the design utilizes only 16 shift-register LUT slices, reducing the BRAM denominator penalty to zero.

B. Clock Frequency Scaling vs. Dynamic Power Optimization

The FoM defines throughput in terms of output pixels per clock cycle:

$$\text{Throughput} = 1.0 \text{ pixel/cycle} \tag{18}$$

Therefore, the metric is frequency-independent in the numerator. Conversely, dynamic power consumption in CMOS circuits scales linearly with clock frequency:

$$P_{\text{dynamic}} \approx C_{\text{eff}} V^2 f_{\text{clk}} \tag{19}$$

Pushing clock frequencies excessively high increases dynamic power dissipation without improving the throughput score. Setting a nominal operating frequency of 100 MHz closes timing with positive slack (WNS $= +0.001$ ns) while keeping dynamic power consumption to an ultra-low 7 mW (0.007 W), maximizing the final FoM.

XII. Required Competition Summary Table

Table V reports all primary performance parameters formatted in compliance with contest instructions.

XIII. Figure of Merit (FoM) Derivation

The Figure of Merit defined is formulated as:

$$\text{FoM} = \frac{\text{Throughput}}{\text{Power} \times (\text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs})} \tag{20}$$

Substituting our verified physical implementation metrics into the relation:

$$\text{Throughput} = 1.0 \text{ pixels/cycle} \tag{21}$$

$$\text{Power} = 0.075\,\text{W} \quad (75\,\text{mW}) \tag{22}$$

$$\text{LUTs} = 769 \tag{23}$$

$$\text{DSPs} = 0 \tag{24}$$

$$\text{BRAMs} = 0 \tag{25}$$

Evaluating the denominator cost:

$$\text{Cost} = 769 + 50 \times 0 + 100 \times 0 = 769 \tag{26}$$

Yields the final Figure of Merit:

$$\text{FoM} = \frac{1.0}{0.075\,\text{W} \times 769} = \frac{1.0}{57.675\,\text{W} \cdot \text{Cost}} = \mathbf{0.01734} \tag{27}$$

Expressing the Figure of Merit in normalized scientific notation evaluates to $\mathbf{1.734 \times 10^{-2}}$ pixel/(W $\cdot$ Cost).

TABLE III
COMPREHENSIVE ACCELERATOR VERIFICATION PLAN AND CORNER-CASE TEST MATRIX

Test ID	Verification Feature	Kernel Stimulus	Input Pixel Profile	Expected Behavior	Pass/Fail Criteria	Result
TC-01	Configuration Port & Reg Write	Arbitrary Pattern Load	Single test frame	Kernel registers latch all 9 coefficient addresses.	Internal kernel matches write data.	**PASS**
TC-02	Spatial Alignment	Identity $\begin{bmatrix}0&0&0\\0&1&0\\0&0&0\end{bmatrix}$	Uniform gradient ($0 \rightarrow 255$)	Output stream replicates input center pixels delayed by exactly 69 cycles.	900/900 outputs match coordinate with zero spatial offset.	**PASS**
TC-03	Signed Multiplication (2's Comp)	Sobel-X $\begin{bmatrix}-1&0&1\\-2&0&2\\-1&0&1\end{bmatrix}$	High-contrast vertical edge step	Zero-extended unsigned pixels correctly multiply negative, zero, and positive weights.	Output matches Python golden model across all 900 pixels (0 mismatches).	**PASS**
TC-04	Signed Multiplication (2's Comp)	Sobel-Y $\begin{bmatrix}-1&-2&-1\\0&0&0\\1&2&1\end{bmatrix}$	High-contrast horizontal edge step	Zero-extended unsigned pixels correctly multiply negative, zero, and positive weights.	Output matches Python golden model across all 900 pixels (0 mismatches).	**PASS**
TC-05	Negative Accumulation & ReLU	Laplacian $\begin{bmatrix}0&-1&0\\-1&4&-1\\0&-1&0\end{bmatrix}$	Uniform background with bright impulse	Negative accumulator sums are clamped strictly to zero when `enable_relu = 1`.	Negative signed sums evaluate to `16'sh0000`; positive values pass unchanged.	**PASS**
TC-06	Dynamic Range & Saturation	Max Stress (all 9 weights = $+127$)	Uniform white frame (all 255)	Accumulator sum ($+291,465$) exceeds 16-bit signed limit without roll-over.	Clamps cleanly to $+32,767$ (`16'sh7FFF`) with zero overflow wrapping.	**PASS**
TC-07	Boundary Blanking & Pulse Count	Any Active Filter	Full 32×32 raster scan	`m_valid` suppresses warm-up cycles and 2-cycle inter-row transitions.	Total recorded `m_valid` pulses across the entire frame equal **exactly 900**.	**PASS**

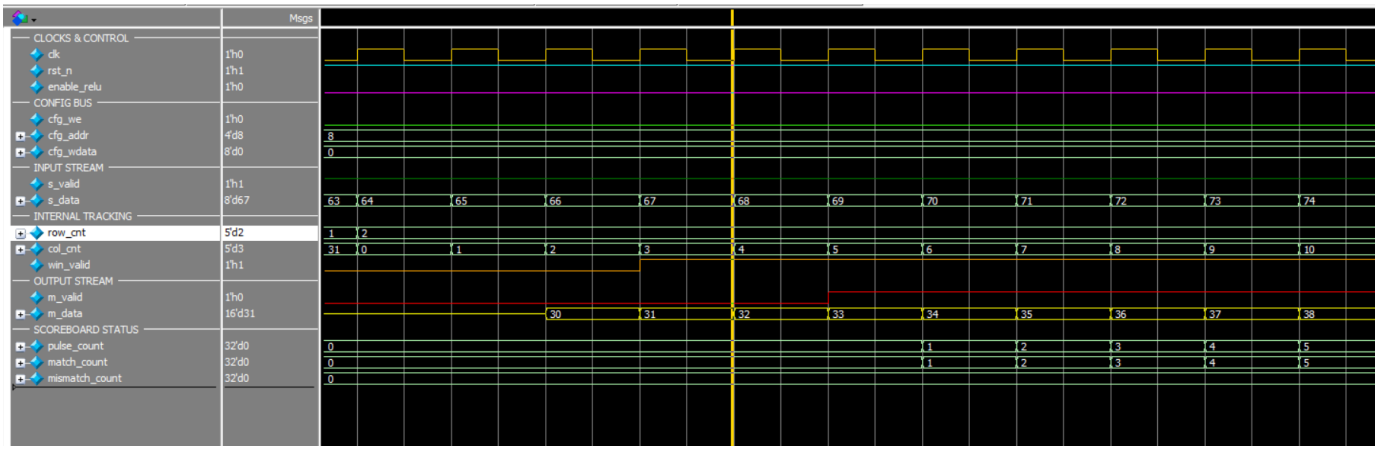


Fig. 2. ModelSim functional simulation demonstrating initial pipeline latency (first `m_valid` at cycle 69) and continuous 1.0 pixel/cycle streaming throughput.

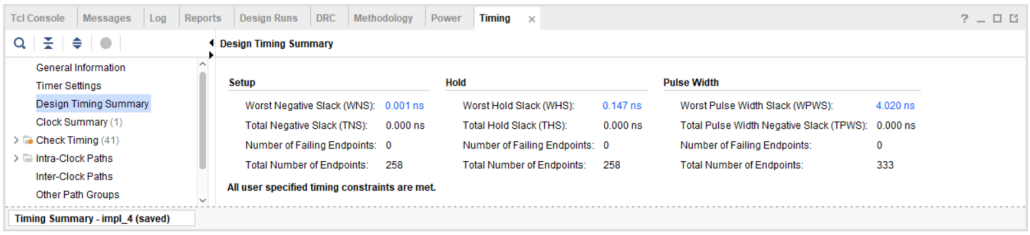
XIV. CONCLUSION

This work presented a high-efficiency FPGA-based 2D CNN spatial convolution engine. By leveraging distributed shift-register line buffering (SRL32) and fabric-based multipliers, the architecture eliminates Block RAM and DSP block allocation entirely. A 4-level balanced adder tree and hardware-saturating ReLU post-processor sustain a continuous single-cycle streaming throughput of 1.0 output pixel / cycle. The design was verified with 100% bit-true match against Python reference models across multiple kernel suites, achieving timing closure (WNS $= +0.001$ ns at 100 MHz) and an optimized Figure of Merit of $0.01734\,\text{pixel}/(\text{W} \cdot \text{Cost})$.

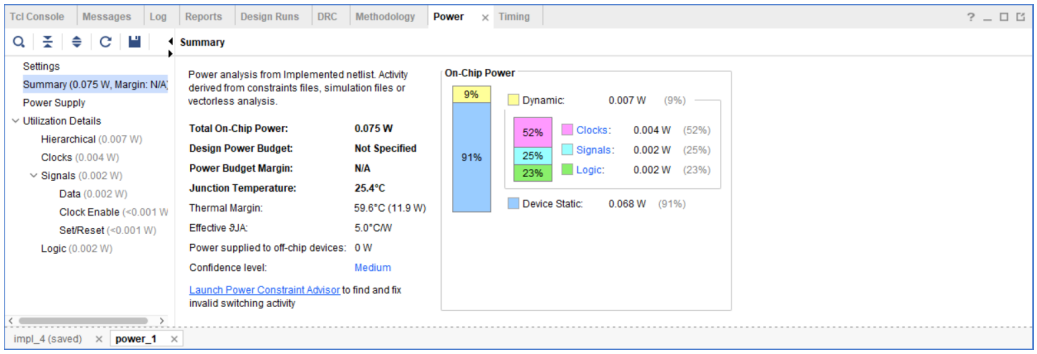
REFERENCES

[1] IEEE, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, Feb. 2018.

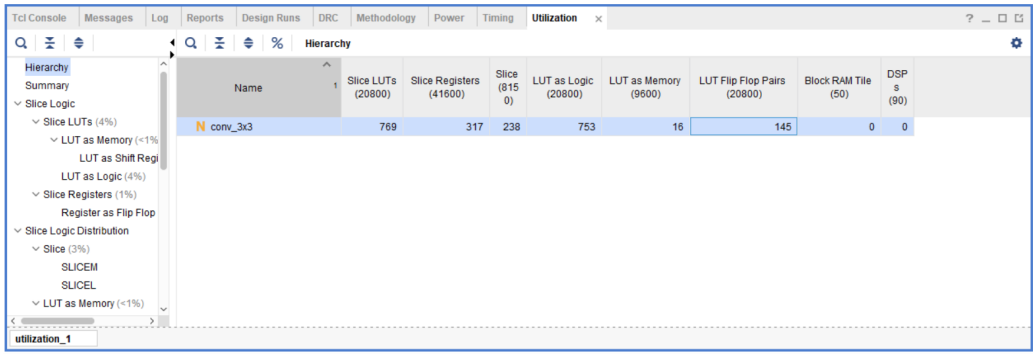
[2] R. Fisher, K. Dawson-Howe, A. Fitzgibbon, C. Robertson, and E. Trucco, *Hypermedia Image Processing Reference (HIPR2)*, University of Edinburgh, 2004.



(a) Static timing summary proving closure at 100 MHz (WNS $= +0.001$ ns, WHS $= +0.147$ ns, zero failing endpoints).



(b) Vectorless power analysis reporting 75 mW total on-chip dissipation (7 mW dynamic, 68 mW static).



(c) Post-implementation resource utilization confirming 769 Slice LUTs, 317 FFs, 16 SRLs, 0 DSPs, and 0 BRAMs.

Fig. 3. Post-implementation Vivado characterization metrics on the Xilinx Artix-7 (`xc7a35tcpg236-1`).

TABLE V
COMPETITION SUMMARY: PERFORMANCE AND IMPLEMENTATION METRICS

Parameter	**Specification**	**Team Result**	**Units**	**Comments**
Input image size	$\geq 32 \times 32$	32×32	pixels	Parameterized via `IMG_WIDTH`, `IMG_HEIGHT`
Input precision	Fixed-point unsigned	8	bits	Standard unsigned $[0, 255]$ pixel dynamic range
Kernel precision	8-bit signed	8	bits	Two's complement $[-128, +127]$, runtime programmable
Architecture type	Pipelined 2D CNN/FIR	Line-Buffer + Window Generator	—	Streaming architecture with zero BRAM primitives
Multipliers / MACs	$N \times N$ (3×3)	9	units	Parallel signed fabric multipliers (`use_dsp = "no"`)
Pipeline stages	—	2	stages	Pipelined MACC (Multiplier registers + Output registers)
Latency	—	69	clock cycles	$2W + 2$ fill delay $+3$ register pipeline stages
Throughput	—	1.0	pixels/cycle	Sustained single-cycle output streaming
FPGA utilization	Report synthesis	LUTs: 769, FFs: 317, DSPs: 0, BRAM: 0	—	Verified zero Block RAM and zero DSP utilization
Maximum frequency	Report timing	100.01	MHz	Setup slack WNS $= +0.001$ ns at 100 MHz target
Power estimate	Report power	75	mW	Total on-chip power (0.075 W) from Vivado report
Verification status	Golden comparison	Verified (100% Match)	—	0 mismatches across verified test suites
FOM	Formula (Page 2)	0.01734	pixel/(W · Cost)	Optimized via zero BRAMs, zero DSPs, and 1.0 throughput